Literature Review
P2P File Synchronization System

Action Learning Project

Backend (Go, C++)

Arihant Jain
Su Huizhe

Frontend and CI/CD (Java, GitHub Actions)

Nizar Soufiya
Shengkang Duan

May 28, 2026

Contents

1 Problem Context

1.1 Scenario and Pain Points

TODO (team): add scenario and pain points.

1.2 Problem Definition

TODO (team): add problem definition.

2 Overview of Existing Work

TODO (team): summarize existing work and systems.

3 Comparison of Approaches

We compare tools by setup effort, server need, and directory sync for mixed files [1–6].

Approach	Setup	Server	Dir	Version	Conflict	Binary Large
Drive/Dropbox	Med	Yes	Yes	Limited	Limited	Quotas
Git/GitHub	High	Hosted	Repo	Strong	Manual	Weak
Syncthing	Low	No	Yes	Limited	Basic	Good
Resilio Sync	Low	No	Yes	Limited	Basic	Good
Our system (hyp.)	Low	No	Yes	Rollback	LWW	Good

Cloud tools need accounts and servers. VCS tools provide history but add workflow steps and handle binaries poorly. Syncthing/Resilio avoid servers but still have limited rollback and conflict handling. Our design targets that gap with account-free LAN sync plus LWW rollback [7]. We expect shorter setup, lower LAN latency, and better recovery; these are hypotheses.

4 Identified Gap

4.1 Limitations of Centralized Solutions

TODO (team): add limits of centralized solutions.

4.2 Limitations of Existing Peer-to-Peer Solutions

TODO (team): add limits of existing P2P solutions.

4.3 The Gap

TODO (team): describe the gap.

5 Positioning Your Project

5.1 Key Differentiators

TODO (team): add key differentiators.

5.2 Design Philosophy

TODO (team): add design philosophy.

6 Research to Application Mapping

- **Zero-configuration peer discovery on LAN**
  - **Research Insight:** mDNS and DNS-SD use multicast queries and service records to find services without a central server [8, 9].
  - **Application in project**: Use Go advertise/browse on LAN so peers can join without accounts or setup.
- **Causal ordering for concurrent updates**
  - **Research Insight:** Logical clocks help detect concurrent edits without a global clock [10].
  - **Application in project:** Track per-file metadata (timestamps + vector-clock style state) before conflict handling.
- **Consistency strategy under failures and partitions**
  - **Research Insight:** CAP shows strict consistency is hard under partitions; weakly connected systems use deterministic rules to converge [7, 11].
  - **Application in project:** Favor availability on LAN; use LWW + backups and resync after reconnection.
- **Conflict-handling evolution path**
  - **Research Insight:** CRDTs can avoid conflicts but add state and complexity [12].
  - **Application in project:** Keep CRDTs for later; use LWW + backup now.
- **Bandwidth-efficient synchronization direction**
  - **Research Insight:** Delta transfer (rsync-style) reduces data sent for large or partial changes [13].
  - **Application in Project:** Start with whole-file transfer, then add block-level diffs for large files and multi-peer scaling.
- **Local change detection primitives**
  - **Research Insight:** OS event APIs (inotify, FSEvents) provide change streams without polling [14, 15].
  - **Application in project:** Use native watchers per OS to capture add/modify/delete events and trigger sync.
- **Content identity and transport**
  - **Research Insight:** SHA-256 gives content identity for integrity checks, and TCP gives reliable ordered delivery [16, 17].
  - **Application in project:** Hash files for change detection and verification, and move data over TCP sockets between peers.

7 Key References

Key sources that ground the design choices:

- **LAN service discovery:** RFC 6762/6763 define zero-configuration discovery and advertisement for local networks [8, 9].
- **Causal ordering:** Lamport's logical clocks establish ordering guarantees for concurrent updates [10].
- **Consistency trade-offs:** Brewer's CAP perspective informs availability-first design under partitions [11].
- **Conflict-free replication path:** CRDTs provide a future option for conflict avoidance at the cost of added complexity [12].
- **Efficient sync:** The rsync algorithm motivates later delta-transfer optimizations [13].
- **Implementation primitives:** SHA-256, file event APIs, and TCP support hashing, change detection, and transport [14–17].

8 Conclusion

The literature suggests: make LAN discovery and sync automatic, and use simple, deterministic conflict rules.

- **Key takeaways.** LAN discovery standards and ordering help collaboration, while availability-first consistency with deterministic conflict handling fits weakly connected teams.
- **Design influence.** We use mDNS/DNS-SD, per-file metadata, LWW + backup, and whole-file transfer as the baseline, with delta-sync and CRDTs reserved for later.
- **Limits in existing tools.** Cloud platforms depend on servers, VCS tools add workflow overhead, and P2P sync tools lack rollback for temporary, multi-owner teams.

Overall, we keep shared-folder UX while adding version safety. The expected advantages (setup time, LAN latency, recovery safety) remain hypotheses to test.

References

[1] Google, "Google drive help: Use drive for desktop," https://support.google.com/drive/, 2026, accessed: 2026-05-28.

[2] Dropbox, "Dropbox help center," https://help.dropbox.com/, 2026, accessed: 2026-05-28.

[3] S. Chacon and B. Straub, *Pro Git*, 2nd ed. Apress, 2014.

[4] GitHub, "Github docs: About repositories," https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories, 2026, accessed: 2026-05-28.

[5] S. Community, "Syncthing documentation," https://docs.syncthing.net/, 2026, accessed: 2026-05-28.

[6] I. Resilio, "Resilio sync documentation," https://help.resilio.com/hc/en-us, 2026, accessed: 2026-05-28.

[7] K. Petersen, M. Spreitzer, D. Terry, M. Theimer, and A. Demers, "Managing update conflicts in bayou, a weakly connected replicated storage system," in *Proceedings of the 15th ACM Symposium on Operating Systems Principles*, 1995, pp. 172–182.

[8] S. Cheshire and M. Krochmal, "Multicast dns," RFC 6762, 2013, accessed: 2026-05-28. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6762

[9] ——, "Dns-based service discovery," RFC 6763, 2013, accessed: 2026-05-28. [Online]. Available: https://www.rfc-editor.org/rfc/rfc6763

[10] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Communications of the ACM*, vol. 21, no. 7, pp. 558–565, 1978.

[11] E. A. Brewer, "Brewer's conjecture and the feasibility of consistent, available, partition-tolerant web services," *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002.

[12] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski, "A comprehensive study of convergent and commutative replicated data types," *INRIA Research Report*, no. RR-7506, 2011.

[13] A. Tridgell and P. Mackerras, "The rsync algorithm," in *Proceedings of the USENIX Annual Technical Conference*, 1996.

[14] M. Kerrisk, "inotify(7) — linux manual page," https://man7.org/linux/man-pages/man7/inotify.7.html, 2024, accessed: 2026-05-28.

[15] Apple, "File system events programming guide," https://developer.apple.com/library/archive/documentation/Darwin/Conceptual/FSEvents_ProgGuide/Introduction/Introduction.html, 2007, accessed: 2026-05-28.

[16] N. I. of Standards and Technology, "Fips pub 180-4: Secure hash standard (shs)," 2015, accessed: 2026-05-28. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf

[17] W. M. Eddy, "Transmission control protocol (tcp)," RFC 9293, 2022, accessed: 2026-05-28. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9293